



Tutorial S3B: Relational Database (Solutions)

Discussion Questions

- 1b Patient (PatientID, Gender, Name, DateOfBirth, Address, ContactNo)
Staff(StaffID, Name, Gender, DateOfBirth, Address, ContactNo, Grade)
Ward(WardNo, Class, NoOfBed, Level, StaffID)

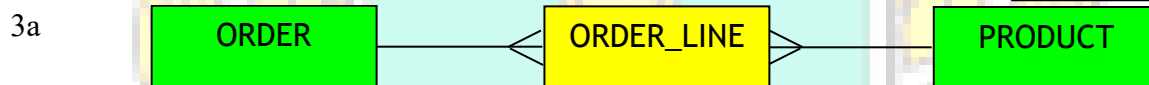
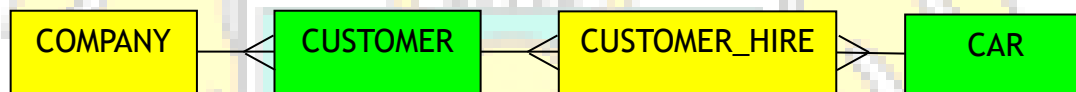


- 2a CUSTOMER (Customer_Number, Customer_Name, Company_Name, Town, Postcode, Car_Registration)
CAR (Car_Registration, Make, Model, Date_Hired, Date_Returned)

- 2b CUSTOMER (Customer_Number, Customer_Name, Company_Name, Town, Postcode)
CUSTOMER_HIRE (Customer_Number, Car_Registration, Date_Hired, Date_Returned)
CAR (Car_Registration, Make, Model)



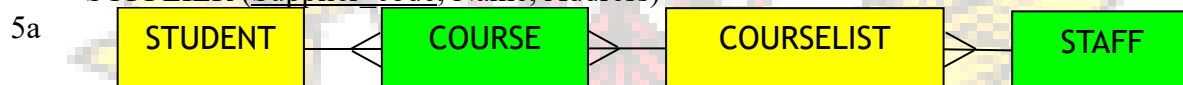
- 2c COMPANY (Company_Name, Town, Postcode)
CUSTOMER (Customer_Number, Customer_Name, Company_Name)
CUSTOMER_HIRE (Customer_Number, Car_Registration, Date_Hired, Date_Returned)
CAR (Car_Registration, Make, Model)



- 3b ORDER (order_number, date)
ORDER_LINE (order_number, product_code, qty_ordered)
PRODUCT (product_code, description, unit_price)



- 4b STOCK (Stock_code, Description, Price)
STOCK_LINE (Stock_code, Supplier_code)
SUPPLIER (Supplier_code, Name, Address)



- 5b STUDENT (studentID, name, class, contactNumber, address)
COURSE (studentID, courseID, dateEnrol)
COURSELIST (courseID, courseName, description, day, duration, staffID)
STAFF (staffID, staffName, roomID, department)

5c From table STAFF, search for staffID that belongs to staffName.
 With the staffID, get all the courseIDs that the staff teaches from table COURSELIST.
 From the courseIDs, get all studentIDs from table COURSE.
 Then get the details of all students that corresponds to these studentIDs from table STUDENT.

6a CUSTOMER (CustID, CustName, Address, TelephoneNo)
 DISH-ORDERS (DishID, CustID, Date)
 DISH (DishID, DishName, ChefID)
 CHEF (ChefID, ChefName, TableNo)



6c From table CHEF, search for ChefID that belongs to the ChefName.
 With the ChefID, get all the DishIDs that the ChefID cooks from table DISH.
 From all the DishIDs, get all CustIDs from table DISH-ORDER.
 Then get all the CustName that corresponds to these CustIDs from table CUSTOMER.

7a Data inconsistency exists when different and conflicting versions of the same data appear in different places. If data is stored more than once, the two or more versions of the data may be different. Ian Smith / Ian Smyth (one of these is spelled incorrectly)

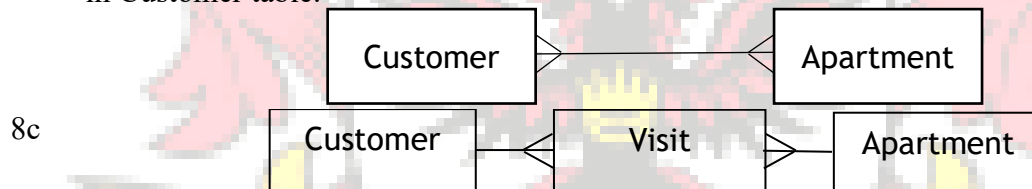
7b Data redundancy is a data organization issue that allows the unnecessary duplication of data within the database. Particulars of Jane Toh appeared 3 times

7c Primary key does not allow duplicate and null values but foreign key allows duplicate and null values.

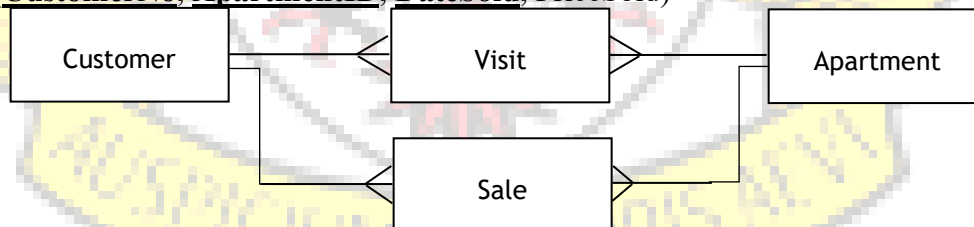
A foreign key is an attribute in a relation of database that serves as the primary key of another relation in the same database. Therefore, foreign key cannot contain values that do not appear in the table that it refers to.

8a Customer (CustomerNo, CustomerName, ContactNumber)
 Apartment (ApartmentID, ApartmentAddress, ApartmentPrice)

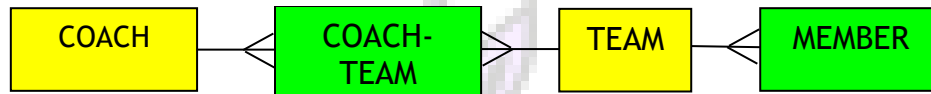
8b Many-to-many relationship. Since a customer can visit many apartments, the records of a customer can appear more than once in Apartment table. An apartment can also be visited by many customers, hence the records of an apartment can appear more than once in Customer table.



8d Sale (CustomerNo, ApartmentID, DateSold, PriceSold)



9a



9bi Primary key: Unique identifier for a record in a table, e.g. CoachID can identify a coach record, TeamID a unique team, MemberID a unique member

9bii Foreign key: A field that matches the primary key of another table, value of field must exist in the primary key of the another table, e.g. memberID must be a valid member of a team and it's memberID also exists in the team.

9biii Composite key: At least two fields that form a primary key to identify a unique record, e.g. a coach's name and date of birth can form a composite key if there is no CoachID, because it is highly unlikely for a name and date of birth to be similar, hence the combine values can form a unique identifier for a coach.

10(a) Insertion anomaly: student details such as student no, student name cannot be inserted until the modules that he's taking is known.

Update anomaly: any update on module information has to be done on all the records which contain the module information, such as module no, module name and lecturer. Otherwise any missed data will cause data inconsistencies.

Deletion anomaly: If any module is completely removed from the records, it may be lost from the database. For example, if Visual Arts is removed, then information about Visual Arts may be lost from the database.

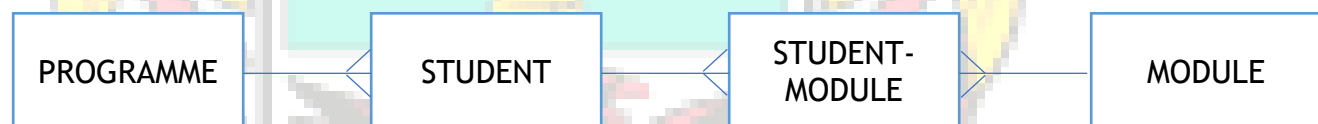
10(b) PROGRAMME (Programme, Programme Duration)

STUDENT (StudentNo, StudentName, Programme)

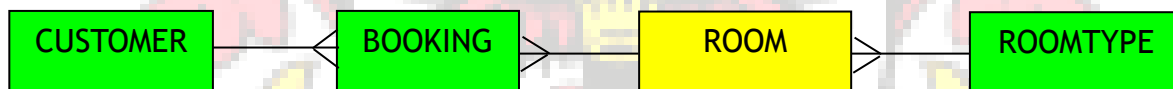
STUDENT-MODULE (StudentNo, ModuleNo)

MODULE (ModuleNo, ModuleName, Lecturer)

10(c) ER diagram:



11a



11b CUSTOMER (CustomerID, Name, Address, Telephone, Email, CreditCardNumber, CreditCardExpireDate)

BOOKING (CustomerID, RoomNumber, ArrivalDate, DepartureDate)

ROOM (RoomNumber, RoomDescription, RoomType)

ROOMTYPE (RoomType, WeekdayRate, WeekendRate)

Possibly create another table CREDITCARD (CreditCardNumber, CreditCardExpireDate, CustomerID) which has a many-to-one relationship with CUSTOMER table.

12ai



- 12aii Tenant (**tenantID**, companyName, director, contactNo, companyAddress, retailType)
 Rental (**tenantID***, **spaceID***, **rentalDateStart**, rentalDateEnd)
 UnitList (**spaceID**, level, unitNo, category*)
 CategoryList (**category**, areaSize, rentalRate)

* foreign key

Examples of tenants in the mall:

CompanyName	level	unitno	RetailType
Bata	1	7-8	Footwear
Hi Tea	1	6	F&B
Giant	1	1-5	Supermarket

Arbitrary values in database:

tenantID	T001	T003	T004
companyName	Bata	Hi Tea	Giant
director	James Bon	Lim Ho	Peter Parker
contactNo	90908080	83737483	85764000
contactAddress	A street	B street	C street
retailType	Footwear	F&B	Supermarket

spaceID	S007	S008	S007	S001	S002	S003	S004	S005
tenantID	T001	T001	T003	T004	T004	T004	T004	T004
rentDateStart	01/01/2011	01/01/2011	01/02/2022	01/02/2011	01/02/2011	01/02/2011	01/02/2011	01/02/2011
rentDateEnd	01/01/2022	01/01/2022	01/02/2025	01/02/2022	01/02/2022	01/02/2022	01/02/2022	01/02/2022

spaceID	S001	S002	S003	S004	S005	S006	S007	S008
level	1	1	1	1	1	1	1	1
unitNo	1	2	3	4	5	6	7	8
spaceCategory	C	C	C	C	C	A	B	B

category	A	B	C
areaSize	<200	200-2000	>2000
rentalRate	40	30	20

- 12b
1. Easier to add in fields when needed. Less effort is needed to add in tables and fields in a database than reprogram the software.
 2. In a customised software, the same information may be held on several different flat files, which not only cause data redundancy but makes updating more time-consuming. In a database system, all the tables are linked which makes updating more efficient as data updated in a table will automatically be updated in other tables that are linked. Redundancy is ***not always completely eliminated***, but it is ***minimised***.
 3. With a database system, users have ***access to information*** that was previously held in separate files in other departments, and sometimes on incompatible systems. The DBMS provides an easy to use ***query*** language that enables users to get instant answers to their queries, rather than having to have a special program written by a programmer, which could take days if not weeks.

